

MASTER_ARCHITECTURE.md - V2

Authoritative project specification transport copy

YouTubeTurboStudio — Master Architecture Specification

Version: 2.0

Purpose: Authoritative technical blueprint for evolving the existing `YouTubeTurboStudio` repository into a secure, extensible, multi-channel YouTube automation platform.

Primary implementation agent: Google Antigravity or another repository-aware coding agent.

1. Project Objective

Upgrade the existing application rather than rebuilding it from scratch.

The final platform must support multiple independent YouTube channels from one application, with channel-specific configuration, credentials, prompts, assets, voices, analytics, and YouTube OAuth identities.

There are two video-generation engines:

- Media Video Engine** — used by **The AI Brief It** and future channels based primarily on stock/video/image media, motion graphics, diagrams, narration, and dynamic captions.
- Animation Engine** — shared by **Kids**, **Elders**, and future character-animation channels.

The normal user workflow should be as close as possible to:

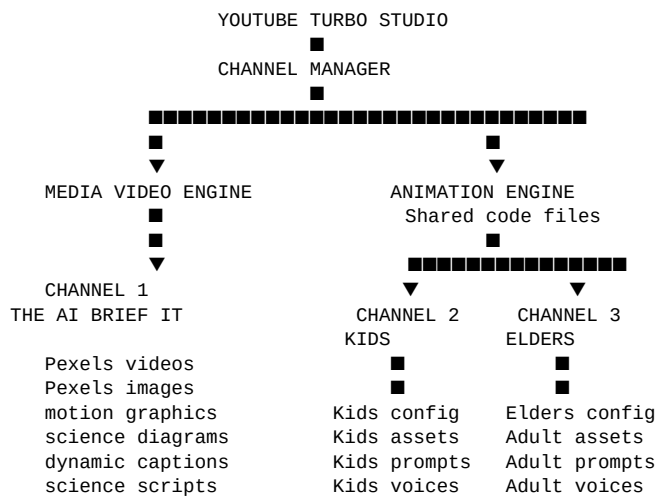
`Select Channel → Optional Topic/Prompt → Select Short/Long → Generate → Review → Publish/Schedule`

The application must also support automatic topic discovery when the user provides no topic.

2. Non-Negotiable Engineering Principles

- Preserve existing working functionality wherever practical.
- Refactor incrementally rather than rewriting the repository.
- Never hard-code exactly three channels.
- New channels must be configuration-driven.
- Never store real API keys, OAuth tokens, client secrets, refresh tokens, or credentials in tracked source files.
- Never commit secrets to Git.
- Every generation job must carry an immutable `channel\_id` from creation through upload.
- Every upload must verify the authenticated YouTube channel before publishing.
- Test videos must never auto-publish to a live channel.
- Kids and Elders must share one Animation Engine implementation.
- The AI Brief It must retain its own Media Video Engine.
- Support both Shorts and long-form videos.
- Run one heavy generation/render pipeline at a time by default on the current laptop, while keeping the job architecture extensible to multiple workers later.
- Optimize for maintainability, security, observability, resumability, and low recurring cost.

3. Target Architecture



The platform must support future Channel 4, Channel 5, etc. without adding a new engine unless the content type genuinely requires one.

4. Suggested Repository Structure

Adapt this structure to the existing repository instead of forcing a destructive migration.

```
YouTubeTurboStudio/
├── AGENTS.md
├── docs/
│   └── MASTER_ARCHITECTURE.md
├── core/
│   ├── channel_registry.py
│   ├── channel_context.py
│   ├── config_loader.py
│   ├── credential_manager.py
│   ├── job_manager.py
│   ├── scheduler.py
│   ├── quality.py
│   └── pipeline_router.py
├── channels/
│   ├── the-ai-brief-it/
│   │   ├── channel.yaml
│   │   ├── prompts.yaml
│   │   └── .env # ignored secret file
│   ├── kids/
│   │   ├── channel.yaml
│   │   ├── prompts.yaml
│   │   └── .env # ignored secret file
│   ├── elders/
│   │   ├── channel.yaml
│   │   ├── prompts.yaml
│   │   └── .env # ignored secret file
│   └── templates/
│       ├── channel.example.yaml
│       └── .env.example
├── engines/
│   ├── media_video/
│   │   ├── pipeline.py
│   │   ├── topic_engine.py
│   │   ├── scriptwriter.py
│   │   ├── critic.py
│   │   ├── fact_checker.py
│   │   ├── visual_director.py
│   │   └── media_manager.py
```

```

■ ■ ■ ■ motion_graphics.py
■ ■ ■ ■ captions.py
■ ■ ■ ■ renderer.py
■ ■ ■ ■ quality.py
■ ■
■ ■ ■ ■ animation/
■ ■ ■ ■ pipeline.py
■ ■ ■ ■ content_writer.py
■ ■ ■ ■ storyboard.py
■ ■ ■ ■ scene_director.py
■ ■ ■ ■ character_engine.py
■ ■ ■ ■ movement_engine.py
■ ■ ■ ■ expression_engine.py
■ ■ ■ ■ lip_sync.py
■ ■ ■ ■ camera.py
■ ■ ■ ■ backgrounds.py
■ ■ ■ ■ props.py
■ ■ ■ ■ captions.py
■ ■ ■ ■ music.py
■ ■ ■ ■ sfx.py
■ ■ ■ ■ renderer.py
■ ■ ■ ■ quality.py
■
■ ■ ■ ■ assets/
■ ■ ■ ■ animation/
■ ■ ■ ■ kids/
■ ■ ■ ■ characters/
■ ■ ■ ■ backgrounds/
■ ■ ■ ■ props/
■ ■ ■ ■ elders/
■ ■ ■ ■ characters/
■ ■ ■ ■ backgrounds/
■ ■ ■ ■ props/
■ ■ ■ ■ music/
■ ■ ■ ■ sfx/
■
■ ■ ■ ■ audio/
■ ■ ■ ■ youtube/
■ ■ ■ ■ auth.py
■ ■ ■ ■ uploader.py
■ ■ ■ ■ analytics.py
■ ■ ■ ■ channel_verifier.py
■
■ ■ ■ ■ runtime/
■ ■ ■ ■ credentials/          # ignored
■ ■ ■ ■ cache/                # ignored as appropriate
■ ■ ■ ■ jobs/
■ ■ ■ ■ output/
■
■ ■ ■ ■ tests/

```

Names may be adjusted to fit the current codebase, but responsibilities and isolation must remain.

5. Channel Registry and Channel Manager

The application must dynamically discover channels from `channels/\*/channel.yaml`.

Do **not** implement `config1.py`, `config2.py`, `config3.py` as the long-term architecture.

Each channel configuration must define at least:

```

id: kids
name: Kids Channel
engine: animation
enabled: true

audience:
  type: children
  age_group: "3-6"

video:
  default_format: short
  allow_short: true
  allow_long: true

```

```
credentials:
  env_file: channels/kids/.env

youtube:
  made_for_kids: true
  oauth_profile: kids

pipeline:
  auto_topic: true
  auto_publish: false
  minimum_quality_score: 92
```

The registry must support:

- list channels
- get channel
- validate channel configuration
- enable/disable channel
- add channel
- edit channel
- remove/deactivate channel safely
- determine engine
- load per-channel settings

Adding a future channel should primarily require configuration, prompts, credentials, and assets—not editing core Python routing logic.

6. Channel Context

Every job must be created with a `ChannelContext` containing at least:

- `channel_id`
- channel name
- engine type
- content strategy
- credential profile
- YouTube OAuth profile
- prompt configuration
- voice configuration
- asset pack
- audience rules
- quality threshold
- output settings

The context must be passed explicitly through services rather than relying on mutable global environment state.

Never use a single global `GEMINI_API_KEY` or YouTube token for all channels when a channel-specific override exists.

7. Credential Architecture

Each channel may have independent API credentials.

Example local secret file:

```
GEMINI_API_KEY=  
GROQ_API_KEY=  
PEXELS_API_KEY=  
PIXABAY_API_KEY=  
ELEVENLABS_API_KEY=  
YOUTUBE_CLIENT_ID=  
YOUTUBE_CLIENT_SECRET=  
YOUTUBE_REFRESH_TOKEN=  
YOUTUBE_CHANNEL_ID=
```

Real `.env` files must be ignored by Git.

Provide a tracked `.env.example` containing empty placeholders only.

Support optional global/default credentials for services that do not need channel isolation, with this precedence:

1. channel-specific credential
2. global/default credential
3. clear configuration error

YouTube OAuth should remain channel-specific.

Required `.gitignore` protection

At minimum protect patterns equivalent to:

```
.env  
.env.*  
!.env.example  
channels/*/.  
channels/*/credentials.json  
channels/*/client_secret*.json  
channels/*/token*.json  
channels/*/oauth*.json  
runtime/credentials/  
credentials/  
secrets/
```

Add secret scanning/pre-commit protection where practical (for example Gitleaks).

If a credential was previously committed, warn the user that `.gitignore` is insufficient and the credential should be rotated/revoked.

8. Separate YouTube OAuth Per Channel

Store OAuth tokens outside tracked source files, e.g.:

```
runtime/credentials/  
■■■ the-ai-brief-it/youtube_token.json  
■■■ kids/youtube_token.json  
■■■ elders/youtube_token.json
```

Before upload:

1. Read ``job.channel_id``.
2. Load that channel's OAuth profile.
3. Query/verify the authenticated YouTube channel identity.
4. Compare it with configured expected channel ID.
5. If mismatch: block upload and show an explicit error.

6. If match: continue.

Never silently fall back to another channel's OAuth profile.

9. UI — Channel Selection

The active channel must be visually obvious at all times.

Provide a channel dropdown/card selector such as:

```
ACTIVE CHANNEL  
[ The AI Brief It ▼ ]
```

When the selected channel changes, the UI must load that channel's:

- engine
- API/provider settings
- YouTube connection
- prompts
- voice
- video defaults
- output options
- content categories
- safeguards
- analytics

The Generate action must display the target channel prominently before starting.

The Publish action must again display and verify the target channel.

10. UI — Save and Persistence Requirements

Every editable settings area must have a clear ****Save**** button.

Examples:

- channel details
- API/provider configuration
- YouTube settings
- prompt settings
- voice settings
- video defaults
- subtitle style
- content categories
- quality thresholds
- automation/schedule settings
- kids age profile
- animation style

Save must:

1. validate values
2. persist non-secret settings safely
3. persist secrets only in secure ignored local storage
4. provide success/error feedback
5. survive application restart
6. never log full secret values

When displaying existing secrets, show masked values only.

11. Pipeline Router

Routing must be configuration-driven:

```
selected channel
  ↓
channel.engine
  ↓
media_video OR animation
```

Examples:

- `the-ai-brief-it` → `media\_video`
- `kids` → `animation`
- `elders` → `animation`

The router must not contain a growing chain of hard-coded channel names.

PART A — MEDIA VIDEO ENGINE

12. The AI Brief It — Preserve and Enhance Existing Code

The current channel must continue to work while quality is improved.

Desired positioning:

****Mind-blowing science and future technology explained visually and clearly.****

Core pillars:

1. Human body and brain
2. Space and extreme science
3. Future science and technology

Avoid random niche drift.

13. Topic Modes

Support two modes:

Automatic Topic Mode

When the topic field is blank or Auto Topic is enabled:

1. generate 15–20 candidates
2. compare with recent channel history
3. score candidates
4. avoid recent duplicates
5. select the best candidate

Suggested scores:

- curiosity
- visual potential
- emotional impact
- novelty
- channel fit
- scientific reliability
- Shorts/long-form suitability
- saturation/repetition risk

User Topic Mode

The user may enter only a topic or prompt, for example:

`What happens to the human body in space?`

The system handles angle, hook, script, storyboard, visuals, voice, captions, metadata, QC, and render automatically.

14. Hook Tournament

For information/science videos, generate multiple hook candidates rather than accepting the first hook.

Generate approximately 8–10 hook variants using structures such as:

- curiosity gap
- surprising fact
- contradiction
- consequence
- challenge
- what-happens-if
- misconception
- unexpected comparison

Score and select the best truthful hook.

Do not use misleading clickbait.

15. Science Script Engine

Scripts must be optimized for clarity, retention, originality, and factual accuracy.

Rules:

- no unnecessary greeting
- hook immediately
- new information/curiosity beat every few seconds where appropriate
- remove filler
- avoid generic conclusions
- do not stretch content to hit a duration
- provide a satisfying payoff
- contextual CTA only when useful
- natural loop only when it improves the video

Support Short and long-form structures separately.

16. Script Critic and Rewrite

Use a second review stage.

Score:

- hook
- curiosity
- clarity
- pacing
- information density
- originality
- emotional impact
- scientific credibility
- visual potential
- payoff
- channel fit

If below threshold, automatically rewrite a limited number of times.

Avoid infinite loops.

Persist original script, feedback, and final script for debugging/analytics.

17. Fact Checking

Extract factual claims and classify them where possible as:

- established
- plausible
- uncertain/speculative

- unsupported

Never invent studies, researchers, statistics, institutions, or consensus.

Use cautious wording for health/medical claims.

Maintain optional internal references/source metadata for important claims.

18. Media Selection Hierarchy

For The AI Brief It, keep production inexpensive.

Preferred hierarchy:

1. relevant Pexels **video**
2. relevant Pexels image with intelligent motion
3. programmatic science diagram/motion graphic
4. branded abstract motion graphic

Do not use unrelated stock simply because a keyword matched.

Score media relevance and quality.

Reject low-resolution, duplicate, watermarked, badly cropped, or clearly irrelevant assets where detectable.

19. Visual Director

Narrative sections must not equal visual scenes.

Create timestamped visual beats. A 30-second Short may have many more visual beats than script paragraphs.

Visual changes should generally occur often enough to maintain attention, commonly around 1.5–3 seconds when appropriate, but should be narration-driven rather than mechanically timed.

Support:

- Pexels clips
- Pexels images
- parallax
- zoom/pan
- kinetic typography
- diagrams
- arrows/highlights
- counters
- timelines
- particle effects
- scientific labels
- transitions

20. Dynamic Caption System

Replace poor/rainbow subtitle styling.

Requirements:

- professional primary caption color
- at most one controlled emphasis/accent color
- typically 2–5 words per caption beat
- usually one line; maximum two when necessary
- readable mobile size
- high contrast
- safe margins from YouTube UI
- subject-aware placement where possible
- accurate synchronization
- optional word-level timing
- restrained animations such as pop/scale/fade/slide
- no long paragraphs
- no random per-word rainbow colors

Run subtitle QC after rendering.

21. Science Voice and Audio

Support provider abstraction.

Examples:

- ElevenLabs when configured
- Microsoft/Edge TTS fallback
- local TTS where appropriate

Support natural pacing, pauses, emphasis, normalization, and consistent loudness.

Background music must not overpower narration.

Support subtle SFX and audio ducking.

PART B — SHARED ANIMATION ENGINE

22. Animation Engine Scope

Build one reusable 2D character-animation engine shared by Kids, Elders, and future animated channels.

Do ****not**** create separate duplicated renderers for Kids and Elders.

The engine must be data/storyboard driven.

The LLM must output validated structured scene/storyboard data, not arbitrary executable rendering code.

23. Standard Storyboard Schema

Define and validate a schema conceptually similar to:

```
{
  "scene": 2,
  "duration": 5.0,
  "background": "garden_day",
  "characters": [
    {
      "id": "benny_bunny",
      "position": "left",
      "action": "walk_right",
      "emotion": "happy"
    }
  ],
  "props": [
    {
      "id": "carrot",
      "count": 5,
      "position": "center"
    }
  ],
  "dialogue": {
    "character": "benny_bunny",
    "text": "Can you count the carrots with me?"
  },
  "camera": {
    "type": "push_in"
  },
  "music_cue": "playful_light",
  "sfx": ["pop"]
}
```

Unknown character IDs/actions/assets must fail validation or trigger a controlled fallback, never arbitrary code execution.

24. Character Engine

Support reusable character definitions with stable identity.

Common actions should include where assets support them:

- idle
- blink
- talk
- smile
- laugh
- sad
- surprised
- walk
- run
- jump
- dance
- wave
- point
- sit
- stand

- sleep
- gesture
- look

The same action API must work for Kids and Elders character packs.

Character consistency across scenes and episodes is mandatory.

25. Movement, Expressions and Camera

Build reusable systems for:

- movement paths
- enter/exit
- walking/running cycles
- simple interactions
- facial expressions
- eye/blink behavior
- gestures
- camera pan
- push-in/pull-out
- zoom
- scene framing
- transitions

Kids profiles may use more energetic movement. Elders profiles may use slower, more natural/cinematic movement.

This must be configuration, not duplicated engine code.

26. Lip Sync

Implement a practical lip-sync pipeline suitable for reusable 2D characters.

Preferred flow:

`TTS audio → alignment/phonemes → mouth shapes → timeline → character render`

Gracefully degrade to simpler mouth-open/mouth-closed animation when detailed phoneme alignment is unavailable.

Do not block all video generation solely because premium lip-sync is unavailable.

27. Asset Packs

Separate engine code from channel assets.

```
assets/animation/
  ■■■ kids/
  ■   ■■■ characters/
  ■   ■■■ backgrounds/
  ■   ■■■ props/
  ■■■ elders/
```

■■■ characters/
■■■ backgrounds/
■■■ props/

Future animated channels should add another asset pack without cloning the animation engine.

PART C — KIDS CHANNEL

28. Kids Content Types

Support at least:

- alphabet
- phonics
- numbers/counting
- colors
- shapes
- animals
- good habits
- original poems
- original rhymes
- narrated rhythmic content
- stories
- moral stories
- bedtime stories
- simple adventures
- educational Shorts

Support configurable age groups, e.g.:

- 2–3
- 4–5
- 6–8

Vocabulary, pacing, complexity, animation, and educational goals must adapt to age profile.

29. Kids Safeguards — Hard Gate

Kids-specific safeguards are mandatory and must not automatically apply to Elders.

Validate before render/publish:

- age-appropriate language
- age-appropriate themes
- safe visuals
- no frightening/disturbing content inappropriate to selected age
- educational correctness

- correct alphabet/phonics representation
- correct counting and displayed quantities where programmatically verifiable
- correct colors/shapes where programmatically verifiable
- no unsafe instructions
- no inappropriate calls to action
- no copyrighted/franchise characters unless the user supplies appropriately licensed assets
- use original or properly licensed character packs
- consistent character identity
- correct Made-for-Kids setting

Failure of critical safeguards must block auto-publish.

30. Kids Original Characters

Create/use original recurring character packs rather than random characters every episode.

Character examples may include original creations such as a bunny, elephant, monkey, owl, etc., but do not imitate protected franchise characters.

Characters should have stable names, appearance, voices, personalities, and animation assets.

31. Kids Stories

The story generator must create original, age-appropriate stories with:

- clear beginning/middle/end
- understandable conflict/problem
- safe resolution
- age-appropriate vocabulary
- character dialogue
- scene descriptions
- expression/action instructions
- optional educational/moral objective
- suitable bedtime pacing when selected

The critic must reject weak, confusing, unsafe, or overly repetitive scripts.

32. Kids Rhymes and Music

Treat rhyme/song generation as a separate content strategy from ordinary stories.

Do not pretend ordinary TTS is high-quality singing.

Initial implementation may support:

- original lyrics
- rhythmic narration
- instrumental/melody generation from reusable/local patterns

- lyric synchronization
- character dance/movement
- sing-along captions

Premium singing providers may be optional integrations later.

Music licensing and commercial-use rights must be respected.

PART D — ELDERS CHANNEL

33. Elders Channel Strategy

Elders uses the ****same Animation Engine**** as Kids.

It must have separate:

- channel config
- prompts
- API credentials
- YouTube OAuth
- adult character pack
- backgrounds/props
- voices
- content categories
- pacing/style
- analytics

Do not inherit kids-specific safeguards except universal platform safety/quality requirements.

Possible categories may include:

- family stories
- emotional stories
- life lessons
- nostalgia
- inspirational stories
- relationship/friendship stories
- village/city life stories
- reflective storytelling

Make these configurable rather than hard-coded.

34. Elders Visual Style

Use the same engine APIs but a different profile:

- adult/older characters

- restrained expressions
- slower movement
- cinematic framing
- mature voices
- less bouncy typography
- realistic or stylized adult environments
- longer story pacing where appropriate

PART E — VIDEO FORMATS

35. Shorts and Long-Form

Every channel configuration must declare whether it supports:

- Shorts
- long-form
- both

Suggested technical defaults:

Shorts

- 1080×1920
- 9:16
- H.264 MP4
- AAC audio
- 30 FPS minimum

Long-form

- 1920×1080
- 16:9
- H.264 MP4
- AAC audio
- 30 FPS minimum

Do not merely crop long-form scenes into Shorts without layout awareness.

Storyboard, caption placement, camera framing, and pacing must be format-aware.

36. Duration Modes

Allow:

- Auto
- Short
- Long

- optional custom target duration

The content strategy determines appropriate duration rather than padding to a fixed length.

PART F — AUTOMATION

37. Generation Modes

Support:

Auto Topic

User selects channel + format and presses Generate. System selects a suitable topic/content idea.

Prompt/Topic Mode

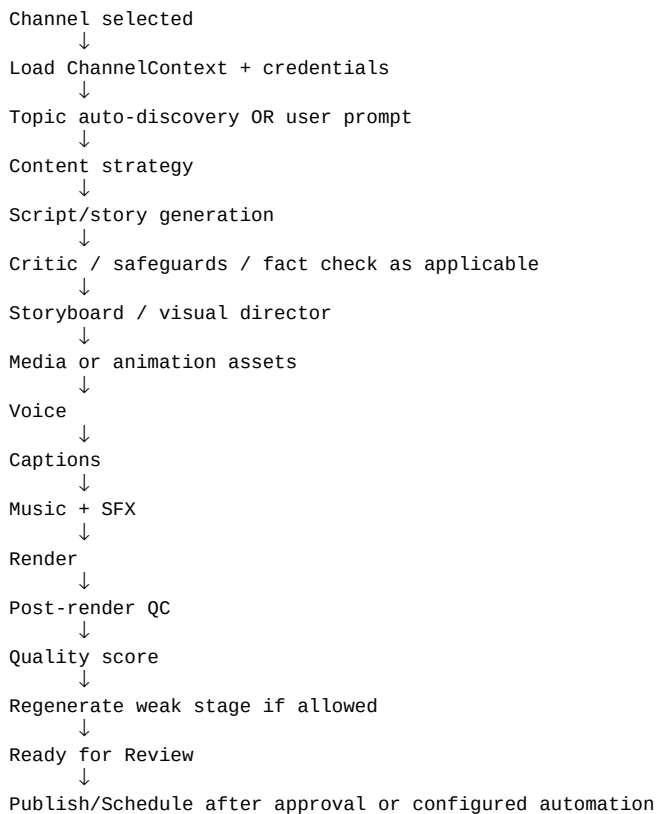
User enters a short topic only. System handles the rest.

Assisted Mode (optional)

System proposes topic/title/script and allows approval/editing before render.

38. Fully Automated Pipeline

Conceptual pipeline:



39. Job Queue and Single Heavy Worker

Design generation as persistent jobs with statuses such as:

- queued
- researching
- scripting
- reviewing
- storyboarding
- acquiring_assets
- generating_audio
- rendering
- qc
- ready
- publishing
- published
- failed

Default to one heavy render worker on the current machine.

The architecture should permit increasing worker count later through configuration without rewriting pipelines.

40. Resumability and Caching

Cache expensive/intermediate outputs:

- research/topic candidates
- scripts
- critic results
- downloaded media
- generated assets
- TTS audio
- alignment data
- storyboards
- rendered scene segments
- metadata

If render fails, resume from the appropriate stage instead of re-running all LLM/API calls.

PART G — QUALITY CONTROL

41. Pre-Publish Quality Score

Create a transparent score out of 100.

Example categories:

- topic/content fit

- hook/opening
- script/story quality
- factual/educational correctness
- storyboard
- visual/media quality
- character consistency where applicable
- captions
- voice
- pacing
- audio
- metadata
- safeguards

Do not fabricate a high score merely to pass.

Suggested policy:

- 90–100: Ready
- 80–89: Good, show improvements
- 70–79: Recommend regeneration
- below 70: block auto-publish unless explicitly overridden

Kids may use a higher minimum and hard safeguard gates.

42. Post-Render QC

Check at minimum:

- output file exists
- correct resolution/aspect ratio
- expected duration
- valid audio
- no clipping where detectable
- no blank/corrupt frames
- no missing scene assets
- no accidental repeated scenes
- captions within safe areas
- no excessively long static section
- no subtitle overflow
- no obvious silent gaps
- metadata exists
- correct channel context

Animation channels additionally check:

- required characters loaded

- character identity consistency
- scene schema validity
- lip-sync/alignment availability or fallback
- prop/count correctness for kids educational scenes where verifiable

PART H — YOUTUBE AND ANALYTICS

43. Publishing

Before publishing show:

- target channel name
- handle/channel ID where available
- title
- format
- Made-for-Kids status
- visibility
- schedule time if any
- quality score

Perform final OAuth/channel verification immediately before upload.

Do not upload failed-QC videos automatically.

44. Made-for-Kids

Kids channel must default to the appropriate Made-for-Kids setting and expose the setting clearly.

Do not automatically apply Made-for-Kids to Elders or The AI Brief It.

The application must use the correct YouTube API field/settings supported by the current integration.

45. Analytics

Where supported by YouTube APIs/scopes, store per-video metrics such as:

- views
- likes
- comments where applicable
- watch time
- average view duration
- average percentage viewed
- subscribers gained
- traffic/retention-related metrics where available

Never fabricate unavailable analytics.

Analytics must be partitioned by channel.

Kids performance must not influence science topic strategy, and vice versa.

46. Learning Loop

Future strategy optimization should use channel-specific historical performance.

Examples:

- better-performing topic pillars
- better hook types
- better durations
- better voice profiles
- better content categories
- better animation pacing

Require sufficient sample size before automatically changing strategy.

Use normalized/time-window metrics where possible so old videos do not unfairly dominate.

PART I — DATA MODEL

47. Persistent Entities

Use the existing persistence mechanism where suitable or introduce a maintainable database layer.

Conceptual entities:

channels

- id
- name
- engine
- enabled
- configuration
- OAuth profile reference

jobs

- job_id
- channel_id
- content_type
- topic/prompt
- format
- status
- timestamps

scripts

- job_id
- draft
- critic feedback
- final script

storyboards

- job_id
- validated scene data

assets

- job_id
- source/type/path/license metadata

renders

- job_id
- output path
- format
- QC result
- quality score

youtube_videos

- job_id
- channel_id
- YouTube video ID
- publish status/time

analytics_snapshots

- channel_id
- video_id
- timestamp
- metrics

settings

Persist non-secret channel/application settings.

Never persist raw secrets in ordinary database fields unless explicitly encrypted and designed for that purpose. Prefer ignored local secret files or OS/keychain facilities.

PART J — PUBLIC GITHUB READINESS

48. Public Repository Requirements

The repository should be safe to publish publicly after the user has removed/rotated any previously exposed secrets.

Tracked repository may include:

- source code
- prompts
- animation components
- sample/config templates
- documentation
- example assets with valid redistribution rights

Must not include:

- API keys
- OAuth refresh tokens
- client secrets
- personal access tokens
- private credentials
- unlicensed commercial assets
- private user data

Provide setup documentation for new users.

49. Add Channel Wizard

Implement or prepare an extensible UI flow:

1. Channel name
2. Internal channel ID/slug
3. Engine selection (`media\_video` or `animation`)
4. Audience/content strategy
5. API/provider configuration
6. YouTube OAuth connection
7. Asset pack
8. Prompt pack
9. voice profile
10. video defaults
11. safeguards/quality threshold
12. Save

After creation, the channel should appear automatically in the dropdown.

No core code modification should be required for a normal new channel using an existing engine.

PART K — IMPLEMENTATION PHASES

50. Phase 1 — Multi-Channel Foundation

Implement first:

- channel registry
- channel context
- dynamic channel dropdown
- per-channel configuration
- per-channel credential loading
- separate YouTube OAuth profiles
- Save/persistence
- pipeline router
- public-repo-safe secret handling
- channel verification before upload
- templates for Science, Kids, Elders
- preserve current science generation

Acceptance Criteria

- All three configured channels appear in UI.
- Switching channel changes loaded configuration.
- Secrets remain isolated.
- Settings persist after restart.
- Science pipeline still runs.
- Kids/Elders can exist before full animation implementation.
- Wrong YouTube OAuth/channel mismatch blocks upload.
- No secret is committed by tests/setup.

Stop and stabilize before Phase 2.

51. Phase 2 — Improve The AI Brief It

Implement:

- topic candidate ranking
- user topic mode
- hook tournament
- improved science script prompt
- critic/rewrite
- fact-checking layer
- Pexels video-first retrieval
- better image selection
- visual director
- motion graphics/diagrams
- professional captions
- improved audio/SFX

- Shorts + long-form support
- QC and quality scoring

Acceptance Criteria

Generate at least one Short and one long-form test locally without publishing.

The Short must demonstrate:

- improved hook
- professional captions
- relevant visual changes
- 1080×1920 output
- no rainbow subtitles
- successful QC

52. Phase 3 — Shared Animation Engine MVP

Implement reusable engine components:

- storyboard schema/validator
- timeline
- character loading
- basic movement
- expressions
- camera
- backgrounds
- props
- voice integration
- simple lip-sync/fallback
- captions
- music/SFX
- 1080p rendering
- QC

Acceptance Criteria

Render two local scenes using the ****same engine****:

1. Kids character walks, talks, gestures, and interacts with a prop.
2. Adult/elder character walks, talks, sits/gestures in a different environment.

No duplicated Kids/Elders renderer implementation.

53. Phase 4 — Kids Channel

Implement:

- age profiles
- content categories
- original recurring characters
- educational templates
- stories
- bedtime stories
- poems/rhymes
- alphabet
- phonics
- numbers
- colors/shapes
- kids safeguards
- Made-for-Kids handling
- Short + long-form
- kids-specific QC

Acceptance Criteria

Generate locally without publishing:

- one alphabet/number educational video
- one story/bedtime video
- one Short

Critical safeguard failures must block publish readiness.

54. Phase 5 — Elders Channel

Implement channel-specific:

- adult prompts
- adult character pack
- mature voice profiles
- adult backgrounds/props
- slower/cinematic animation profile
- story categories
- Short + long-form

Reuse Phase 3 Animation Engine.

Acceptance Criteria

Generate one local adult/elder animated story and one Short using the same engine code as Kids.

55. Phase 6 — Automation, Scheduling and Analytics

Implement:

- persistent job queue
- auto-topic schedules
- one-heavy-worker default
- review queue
- scheduling
- analytics ingestion
- per-channel dashboards
- channel-specific learning loop
- retry/resume behavior

Do not enable unattended live publishing by default until explicitly configured by the user.

PART L — TESTING

56. Required Tests

Add focused automated tests where practical for:

- channel discovery
- config validation
- channel switching
- credential precedence/isolation
- secret masking
- settings persistence
- pipeline routing
- job channel immutability
- OAuth profile selection
- YouTube channel verification
- storyboard schema validation
- unknown character/action rejection
- Kids safeguard rules
- video format selection
- QC rules
- upload blocking on failed QC

Add integration tests for both engines using mocked external APIs where appropriate.

Never make live uploads during automated tests.

PART M — OBSERVABILITY

57. Structured Logging

Use structured, concise logs such as:

```
[CHANNEL] kids selected
[TOPIC] auto topic generated
[SCRIPT] draft complete
[CRITIC] score 8.2 – rewrite required
[STORYBOARD] 12 scenes validated
[VOICE] complete
[RENDER] 1080x1920 / 34.2 sec
[QC] 93/100
[YOUTUBE] channel verified
[JOB] ready for review
```

Never log full API keys/tokens.

PART N — USER CONFIGURATION AFTER CODE GENERATION

58. Mandatory Antigravity Completion Report

After each implementation phase, Antigravity must explicitly tell the user:

1. What files were created.
2. What files were modified.
3. What dependencies were added.
4. What database/schema migrations are required.
5. What environment variables/API keys the user must configure.
6. Exact local secret file paths.
7. How to connect each YouTube channel via OAuth.
8. How to use Save buttons/settings.
9. How to select a channel.
10. How to generate a Short.
11. How to generate a long-form video.
12. How to run tests.
13. How to run the application.
14. How to add a future channel.
15. How to add new character/background/prop assets.
16. Known limitations or deferred work.
17. Security notes.

Never leave the user guessing which keys or setup values must be supplied.

PART O — ANTIGRAVITY WORKING RULES

59. Repository Inspection

Before implementing a phase:

- inspect the current repository
- identify existing modules that can be reused
- identify existing tests
- identify current configuration/secret handling

- identify current YouTube integration
- identify current renderer and media flow

Do not repeatedly rescan unrelated files within the same phase.

60. Credit-Efficient Development

To reduce agent quota usage:

- implement one phase at a time
- inspect only relevant files after initial architecture understanding
- make the smallest maintainable change set
- avoid unrelated refactors
- avoid unnecessary browser research
- run focused tests first
- do not regenerate expensive test videos after every trivial code change
- reuse cached/intermediate assets
- commit stable milestones

61. Git Workflow

Never develop directly on `main`.

Recommended branches:

```
feature/multi-channel-architecture
feature/science-quality-v2
feature/animation-engine
feature/kids-channel
feature/elders-channel
feature/automation-analytics
```

Or use one integration feature branch with milestone commits if that better matches the user's workflow.

Before destructive changes, ensure work is committed.

PART P — FINAL SUCCESS CRITERIA

62. Platform Success Criteria

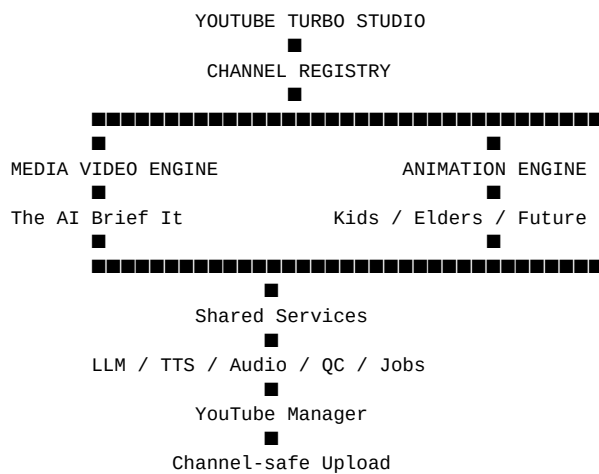
The architecture is considered successful when:

- One application manages multiple channels.
- Channel selection is obvious in the UI.
- Every channel has isolated configuration and credentials.
- Each channel uploads only to its intended YouTube account.
- The AI Brief It uses the enhanced Media Video Engine.
- Kids and Elders share one Animation Engine.
- Kids has hard age/education/safety safeguards.

- Elders does not inherit kids-specific rules.
- User can provide only a topic or use auto-topic mode.
- User can generate Shorts and long-form videos.
- Settings have persistent Save functionality.
- Render/QC failures do not silently publish.
- The codebase is safe to make public after credential rotation/secret-history cleanup.
- Adding another channel using an existing engine is primarily configuration/assets, not new core code.
- The default workflow remains suitable for one heavy pipeline at a time on the current machine.

63. Final Product Vision

`YouTubeTurboStudio` should evolve from a single-channel video generator into a reusable multi-channel automation platform:



The application should automate as much creative and technical work as possible while keeping channel identity, credentials, safeguards, quality controls, and publishing boundaries explicit and safe.

64. Instruction to Coding Agent

Treat this file as the authoritative architecture specification.

When asked to implement a phase:

1. Read `AGENTS.md`.
2. Read this file.
3. Inspect the current repository state.
4. Determine what is already implemented.
5. Implement only the requested phase unless a prerequisite is genuinely required.
6. Preserve working functionality.
7. Run tests.
8. Do not publish test videos.
9. Do not expose or commit secrets.
10. Provide the mandatory completion/configuration report.

38. THREE CONTENT CREATION MODES (MANDATORY UI ARCHITECTURE)

Every channel must expose exactly three primary content creation options in the Studio UI. These options are shared platform behavior and must work with both the Media Video Engine and the Animation Engine.

38.1 Option 1 - Long Video

UI label: `Long Video`

Purpose: generate a new standalone long-form video for the currently selected channel.

Flow:

1. User selects the active channel.
2. User selects `Long Video`.
3. User may enter one topic/prompt OR leave the topic blank and enable Auto Topic.
4. The channel-specific content planner generates/optimizes the topic.
5. The correct channel engine generates the long video.
6. Run channel-specific QC and safety checks.
7. User reviews/approves when review mode is enabled.
8. Upload to the currently selected YouTube channel using only that channel's OAuth profile.
9. Capture and persist the returned YouTube video ID, canonical URL, title, publish state, duration, thumbnail, and content-family metadata.
10. After a successful long-video generation (and especially after upload), this video becomes eligible as a source for `Shorts - Link Long Video`.

The system must support both generated-but-not-yet-published long videos and already-published long videos. If a linked Short is to contain the public long-video URL, the parent long video must be uploaded first so its URL exists.

38.2 Option 2 - Shorts

UI label: `Shorts`

Purpose: generate an independent standalone Short that is NOT linked to any long video.

Rules:

- No parent video is required.
- No long-video URL is inserted automatically.
- The Short receives its own topic, hook, script, storyboard/visual plan, title, description, hashtags, QC score, and upload record.
- User may enter one topic/prompt OR leave it blank for Auto Topic.
- The currently selected channel determines prompts, engine, credentials, safeguards, voice, assets, metadata, and YouTube destination.
- This mode must remain available even if the channel has never generated a long video.

38.3 Option 3 - Shorts - Link Long Video

UI label: `Shorts - Link Long Video`

Purpose: generate one or more purpose-built Shorts that are explicitly related to a selected long video and funnel interested viewers to that long video.

Availability rules:

- This option is disabled when the selected channel has no eligible long videos.
- It becomes enabled automatically as soon as at least one eligible long video exists for the selected channel.
- The long-video selector must show ONLY long videos belonging to the currently active channel.
- Never allow a Kids Short to link to an Elders or Science long video, or any other cross-channel relationship.

UI behavior after enabling:

- Show a searchable/selectable long-video dropdown/card list.
- Display title, thumbnail when available, duration, generation/publish status, YouTube status, and publish date.
- Allow source filters such as `Generated in Studio` and `Published on YouTube` when supported.
- Allow selecting number of Shorts to create.
- Allow Short duration profile (Auto, 15-30 sec, 20-40 sec, etc.).
- Default generation strategy should be `Purpose-built linked Short` rather than blindly clipping a random segment.
- Optional strategies may include `Extract Clip` and `Hybrid`, but purpose-built linked Short is the preferred default.

Linked Short generation flow:

1. Select active channel.
2. Select `Shorts - Link Long Video`.
3. Select one eligible parent long video.
4. Load the parent's stored script/transcript/storyboard/metadata where available.
5. Analyze the parent and identify the strongest self-contained Short concepts.
6. Rank candidate Short concepts for hook strength, standalone value, visual potential, originality, and connection to the parent.
7. Generate unique Short scripts/hooks rather than repeating the same Short several times.
8. Render using the active channel's engine.
9. Run QC.
10. Persist `parent\_content\_id`, `content\_family\_id`, `parent\_youtube\_video\_id`, and `parent\_youtube\_url` where available.
11. When uploading, automatically include the parent long-video link in the Short description when the public URL exists.
12. Where supported by YouTube/API capabilities, also associate the Short with the related long video using supported related-video functionality.
13. Add a concise contextual CTA such as `Watch the full story` / `Watch the full explanation` when appropriate for the channel.

A linked Short must still provide useful standalone value. Do not create low-value teaser spam whose only purpose is to say `watch the full video`.

38.4 Required UI layout

The content creation area should make the three choices visually obvious:

CONTENT TYPE

[Long Video]
[Shorts]
[Shorts - Link Long Video]

When `Long Video` is selected:

```
Topic / Prompt: [ _____ ]
[ ] Auto Topic
Duration: [Auto v]
[ Generate Long Video ]
```

When `Shorts` is selected:

```
Topic / Prompt: [ _____ ]
[ ] Auto Topic
Duration: [Auto v]
[ Generate Short ]
```

When `Shorts - Link Long Video` is selected:

```
Parent Long Video: [ Select Long Video v ]
Number of Shorts: [ 1 / 2 / 3 / ... ]
Generation Mode: [ Purpose-built / Clip / Hybrid ]
[x] Add parent long-video URL to description
[x] Add contextual Watch Full Video CTA
[x] Keep all Shorts in the same content family
[ Generate Linked Shorts ]
```

If there are no eligible long videos:

```
Shorts - Link Long Video [DISABLED]
Generate or import a Long Video first to enable this option.
```

38.5 Content relationship data model

The persistent content model must support at minimum:

```
content_id
channel_id
engine_id
content_type           # LONG or SHORT
relationship_type      # STANDALONE, PARENT, LINKED_SHORT
content_family_id
parent_content_id
source_content_id
topic
title
script_or_transcript
status
quality_score
youtube_video_id
youtube_url
published_at
created_at
updated_at
```

For a standalone Short:

```
content_type = SHORT
relationship_type = STANDALONE
parent_content_id = null
```

For a long video:

```
content_type = LONG
relationship_type = PARENT or STANDALONE_LONG
```

For a linked Short:

```
content_type = SHORT
relationship_type = LINKED_SHORT
parent_content_id = <selected long content_id>
content_family_id = <same family as parent>
```

38.6 Parent URL metadata behavior

For linked Shorts, the publishing engine must build metadata only after resolving the parent long video's current YouTube state.

If the parent is published:

- Insert the canonical parent URL into the Short description.
- Use a channel-specific CTA template.
- Store the exact URL used in the upload record.

If the parent is generated but not yet published:

- The linked Short may be generated and queued.
- Do NOT invent a YouTube URL.
- Hold the Short upload until the parent is uploaded and a real video ID/URL is available, unless the user explicitly disables parent-link insertion.
- Once the parent uploads, resolve the URL and finalize Short metadata automatically.

38.7 Channel-specific examples

The AI Brief It

Long: `What Sleep Deprivation Does to Your Brain`

Linked Shorts may cover:

- microsleeps
- memory impairment
- emotional regulation

The Short description automatically links to the full science video.

Kids

Long: `Benny Bunny Learns Numbers 1-10`

Linked Shorts may cover:

- count five carrots
- what comes after seven
- find number nine

Kids-specific safeguards remain mandatory. Do not rely on comments/pinned comments as the linking strategy for Made-for-Kids content.

Elders

Long: `The Grandfather Who Returned Home After 40 Years`

Linked Shorts may cover:

- the return to the village
- a major emotional reveal
- a meaningful life lesson

The same Animation Engine is reused; only channel configuration, assets, prompts, voices, and style differ.

38.8 Existing YouTube long videos

Where supported by the configured YouTube APIs/scopes, allow the user to select an existing long-form video from the currently authenticated YouTube channel even if it was not originally generated by YouTubeTurboStudio.

Import enough metadata to create a local parent content record. If transcript/script data is unavailable, require an available transcript/caption source or user-provided context before generating content that claims to summarize the video. Never fabricate the parent's content.

38.9 Shared implementation requirement

Do NOT implement this three-mode UI and relationship logic separately for Science, Kids, and Elders.

Implement it once in shared modules such as:

```
core/content_planner.py
core/content_repository.py
core/content_family.py
core/publishing.py
ui/content_creation.*
```

Then route rendering to:

- `media\_video` engine for The AI Brief It
- `animation` engine for Kids
- `animation` engine for Elders

38.10 Acceptance criteria

The feature is complete only when all of the following pass:

1. UI shows exactly the three primary options: Long Video, Shorts, Shorts - Link Long Video.
2. Standalone Shorts can be generated without any parent.
3. Linked-Short option is disabled when no eligible long video exists.
4. It enables automatically after an eligible long video exists.
5. Parent selector never shows videos from another channel.
6. Linked Short stores parent/content-family relationships persistently.
7. Published parent URL is inserted automatically into linked Short description.
8. No fake URL is created when the parent is unpublished.
9. Queued linked Short can resolve the parent URL after the parent is published.
10. Switching channels immediately changes the eligible parent-video list and credentials.
11. The feature works with both Media Video Engine and Animation Engine.
12. Kids safeguards continue to run for Kids linked Shorts.
13. Long and Short generation support both manual topic/prompt and Auto Topic where applicable.
14. Tests cover cross-channel isolation and prevent accidental cross-linking.
15. No test upload is published without explicit test approval/configuration.